

SWOT Analysis — RwaSport / RNSP

A mobile-first platform to digitize all of Rwandan sport — professional leagues, 20+ federations, a live Match Center, a school-competitions module (AKC3/Amashuri), and racing — with a monetization layer. React PWA + Express / Prisma / PostgreSQL. Grounded in a full code sweep and live verification.

● App verified running

42 data models

~67 frontend pages

3 locales (EN/RW/FR)

20 sports · 24 federations

Readiness ~7/10

8
STRENGTHS

8
WEAKNESSES

6
OPPORTUNITIES

6
THREATS

StrengthsINTERNAL · POSITIVE	
AREA	EVIDENCE
Exceptional domain breadth	42 Prisma models, 34 enums, 6 domains. Sport <i>polymorphism</i> — team sports use Fixtures/Lineups, cycling/athletics use Race/Classification. 14 match-event types, formations, xG, live state, discipline → suspension automation.
Real, realistic data	20 real sports, 24 real federations (FERWABA, FERWABA...), real RPL clubs (APR, Rayon Sports, Police FC), real Tour du Rwanda stages with real riders. Idempotent upsert-based seeds.
Solid auth & security core	JWT + rotating refresh tokens (SHA-256 at rest), bcrypt cost-12, Helmet, CORS allowlist, tiered rate-limiting, httpOnly/SameSite=strict cookies, Zod on mutations, MIME/size-gated uploads via Sharp → Cloudinary.
Feature-complete frontend	~67 pages, wired to the live API (home, fixtures, standings, live counters confirmed rendering real data). 60+ lazy-loaded routes across public / admin / team / reporter / AKC3 portals.
Exemplary design system	DESIGN.md is production-grade: token-only color with measured WCAG ratios, self-hosted Archivo (avoids CDN RTT on Rwandan

AREA	EVIDENCE
	mobile), tabular numerals, no shadows/gradients, dark-first for outdoor use.
Mature i18n	Three <i>complete</i> locales (EN/RW/FR, ~60 KB each) + date-fns localization + enum translation.
Clean architecture	Routes → controllers → services; Zustand + TanStack Query; Axios auto-refresh interceptor; Zod-validated env at boot; pooler-safe Prisma; PWA deliberately never caches <code>/api</code> (live scores never stale).
Clever offline demo	Self-contained <code>demo/app</code> with a mock adapter + committed <code>dist</code> — zero-install pitch capability; no backend needed.

Weaknesses INTERNAL · NEGATIVE		
AREA	EVIDENCE	SEV
No working quality gates	<code>npm run typecheck</code> fails (552 backend / 395 frontend <code>tsc</code> errors) and <code>npm run lint</code> is broken (no eslint config). <i>Nuance:</i> most backend errors are <code>TS2451 "cannot redeclare"</code> — files are CommonJS, so <code>tsc</code> treats them as global scripts. The app runs fine via <code>tsx</code> the typecheck script is misconfigured, not 552 real bugs. But frontend store typing is genuinely soft. Net: no functioning type/lint gate.	High
Almost no tests	Backend: 1 smoke file (~13 read-only cases). Frontend: zero . No unit / integration / e2e, no coverage.	High
No CI/CD	No <code>.github/workflows</code> no pre-commit hooks. Nothing blocks a broken commit; deploys aren't gated.	High
Modeled-but-unbuilt features	Schema + seed exist but no API routes for Transfers, Team Officials, Registrations review, manual Suspensions. Racing is read-only (no result entry).	Med
Payments are mock	<code>payments.controller</code> hardcodes a fake gateway — no real Flutterwave / webhook / signature. Subscription revenue is a stub, so monetization isn't live.	Med

AREA	EVIDENCE	SEV
Missing DB indexes	Only 1 explicit <code>@@index</code> (Race.sportId). Hot FKs <code>Fixture.leagueId</code> <code>Standing.leagueId</code> <code>MatchEvent.fixtureId</code> <code>unindexed</code> → table scans at scale.	Med
Demo/real duplication	~30–40% of the frontend mirrored between <code>demo/app</code> and <code>apps/frontend</code> sync is a manual "port it over" workflow → drift risk.	Med
Tech debt / cleanup	Dead <code>Team.jerseyHome/jerseyAway</code> <code>clubColors.js</code> hardcoded stopgap; orphaned <code>/api</code> dir; no root README; 77 KB <code>PROMPT.md</code>	Low

CORRECTION TO THE SWEEP
The exploration flagged `.env` and `Admin_Credentials.pdf` as "committed secrets." **Both are false** — verified against `git ls-files`: `.env` is properly gitignored (only `.env.example` is tracked) and the PDF is untracked. The real related risk is that secrets live in a **OneDrive-synced** `.env` (cloud-synced local exposure), not a git leak.

Opportunities	EXTERNAL · POSITIVE	
➤ First-mover in a paper-based market. Positioned as Rwanda's first digital sports ecosystem; most leagues still run on spreadsheets. Whole-lifecycle coverage (federation → pro league → school) is a real moat. ➤ AKC3 youth pipeline. Digitizing school sport trains thousands of future athletes/managers on the platform young, and opens the youth-demographic sponsorship market. ➤ Low-OpEx, consumption-based infra. Vercel / Supabase / Cloudinary / Pusher scale with usage and keep off-season costs down. ➤ Three-stream revenue. Verification SaaS + match-day/banner ads + transfer fees — once real payments land. ➤ Premium ad inventory. The live-reporting flow (match-day sponsorship) is high-engagement and directly monetizable. ➤ Fast path to "operable." Finishing the CRUD gaps, wiring real payments, and adding indexes would move it from impressive demo to sellable product in a few sprints.		

- **Vendor single-points-of-failure.** Supabase (all data), Pusher (live scores — the sellable moment collapses if down), Cloudinary (crests + verification docs), Vercel (deploys). No documented DR / failover / multi-region — riskier given Rwandan connectivity variability.
- **Trust-dependent revenue.** The B2B model hinges on accurate verification and standings; that logic has historically been fragile (score double-counting, winner-not-recorded, over-strict doc rules) — reportedly fixed, but it's the fragile surface.
- **Adoption risk.** No documented federation partnerships or pilots; verification fees face price sensitivity; federations may resist cloud data sovereignty.
- **Key-person / small-team risk.** Domain knowledge is concentrated; tight coupling (one deploy, one DB) amplifies it.
- **Stale issue register vs. reality.** The tracked register lists admin/team/live-reporting as broken stubs, but recent work and the live run show these built and serving real data — treat the register as lagging and reconcile it.
- **Real-time cost exposure.** Pusher is pay-per-message; a popular live match (goals/cards × thousands of concurrent fans) could spike costs unpredictably.

Bottom line

A genuinely impressive, feature-rich, **running** product (~7/10 production readiness) with a soft engineering-hygiene layer underneath. The vision and breadth are the standout strengths; the gaps are all in gates and finishing, not architecture.

1 Make payments real

Integrate a live gateway + webhook/signature — unblocks the entire revenue model.

2 Fix typecheck + add ESLint config + minimal CI

GitHub Actions running typecheck and backend smoke tests. Cheap, high-leverage.

3 Add DB indexes on hot foreign keys

One migration; protects against a scale cliff on fixtures/standings/events.

4 Fill the CRUD gaps

Transfers, team officials, registrations review, racing result entry.

5 Add frontend e2e tests

Playwright for login → team registration → live reporting.

6

Reconcile the issue register & move secrets off OneDrive

Sync the tracker to current reality; put `.env` secrets in a proper secret store.